Predicting Election Results Using Social Media Sentiment Analysis

A Hybrid Approach Combining VADER and BERT Models

A Project Report

Submitted by

Project Contributor

December 14, 2025

Table of Contents

CHAPTER 1: INTRODUCTION **3**

1.1 Background and Motivation 3

1.2 Project Objectives and Scope 4

1.3 Report Structure 4

CHAPTER 2: LITERATURE REVIEW **5**

2.1 Introduction 5

2.2 Sentiment Analysis in Political Science 5

2.3 Methodologies in Sentiment Analysis 6

2.3.1 Lexicon-Based Approaches: VADER 6

2.3.2 Machine Learning Approaches: BERT 7

2.4 Hybrid Models and Research Gaps 8

CHAPTER 3: PROBLEM FORMULATION **9**

3.1 Problem Statement 9

3.2 Research Questions 9

3.3 Assumptions and Constraints 10

CHAPTER 4: PROPOSED WORK **11**

4.1 Overview of the Proposed Methodology 11

4.2 Justification for Hybrid Model 12

CHAPTER 5: SYSTEM DESIGN **13**

5.1 System Architecture 13

5.2 Module Descriptions 14

CHAPTER 6: IMPLEMENTATION **15**

6.1 Environment and Libraries 15

6.2 Data Loading and Preprocessing 16

6.3 Language Detection 17

6.4 Hybrid Sentiment Analysis Engine 18

6.4.1 VADER Sentiment Scoring 18

6.4.2 BERT Sentiment Scoring 19
6.4.3 Final Sentiment Aggregation 20
6.5 Vote Share Simulation and Prediction 20
6.6 Output Generation 21
CHAPTER 7: RESULT ANALYSIS 22
7.1 Analysis of Sentiment Scores 22
7.2 Comparative Analysis of VADER and BERT 23
7.3 Keyword Frequency Analysis 24
7.4 Predicted Vote Share Analysis 25
CHAPTER 8: CONCLUSION, LIMITATION, AND FUTURE SCOPE 26
8.1 Conclusion 26
8.2 Limitations of the Study 26
8.3 Future Scope 27
REFERENCE 28
CONTRIBUTION OF PROJECT 29

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

In the 21st century, the confluence of social media and political discourse has created an unprecedented landscape for public opinion analysis. Platforms like Twitter, Facebook, and others have evolved from simple social networking sites into vibrant, albeit chaotic, arenas for political debate, campaigning, and citizen journalism. Billions of users worldwide generate vast quantities of data daily, expressing opinions, sentiments, and reactions to political events, candidates, and policies. This digital deluge presents both a challenge and an opportunity for political scientists, data analysts, and campaign strategists.

Traditionally, predicting election outcomes relied on methods such as public opinion polls, historical voting patterns, and expert analysis. While valuable, these methods are often costly, time-consuming, and subject to biases like social desirability and sampling errors. The rise of computational social science offers a complementary, and potentially more dynamic, approach. By analyzing the sentiment of social media data, it may be possible to gauge the "pulse" of the electorate in real-time, providing a granular and immediate measure of public opinion (Tumasjan et al., 2010).

Sentiment analysis, or opinion mining, is a field of Natural Language Processing (NLP) dedicated to extracting and quantifying subjective information from text data. The core idea is to determine the emotional tone behind a series of words, used to understand attitudes, opinions, and emotions expressed in online mentions. When applied to the political domain, sentiment analysis can help answer critical questions: How do voters feel about a particular candidate? Is a policy announcement being received positively or negatively? Which campaign messages are resonating most with the public?

This project is motivated by the potential of sentiment analysis to serve as a modern tool for electoral prediction. However, the task is fraught with complexity. Social media text is often short, informal, and filled with slang, sarcasm, and context-dependent language, making accurate sentiment classification a significant challenge. This study aims to address this challenge by developing and evaluating a robust pipeline for predicting election outcomes based on social media sentiment, leveraging a hybrid approach that combines the strengths of different sentiment analysis techniques.

1.2 Project Objectives and Scope

The primary objective of this project is to design, implement, and evaluate a computational system that predicts simulated election vote shares based on the sentiment analysis of Twitter data. To achieve this, the project will pursue the following specific goals:

1. To implement a data processing pipeline for cleaning and preparing raw tweet data for analysis.
2. To apply and compare two distinct sentiment analysis methodologies: a lexicon-based approach (VADER) and a state-of-the-art transformer-based model (multilingual BERT).
3. To develop a hybrid sentiment score by aggregating the outputs of both models to create a more robust sentiment metric.
4. To create a simple linear model that translates the aggregated sentiment score into a simulated "vote share" metric.
5. To analyze the results, including the distribution of sentiment, key topics of discussion, and the performance of the predictive model.

The scope of this project is defined by the provided implementation script. The analysis is conducted on a dataset of 5,000 tweets from a `tweets.csv` file. The project focuses on the technical implementation of the sentiment analysis pipeline and the subsequent predictive model. It does not involve real-time data scraping or validation against actual election results. Instead, it serves as a proof-of-concept for the methodology, simulating a vote share to demonstrate the potential application of the sentiment scores. The analysis is multilingual to the extent that the chosen BERT model supports it, with VADER being applied specifically to English-language tweets.

1.3 Report Structure

This report is structured into eight chapters. **Chapter 2** provides a literature review of sentiment analysis, its application in political science, and the theoretical underpinnings of VADER and BERT. **Chapter 3** formulates the specific problem this project addresses. **Chapter 4** outlines the proposed work and methodology. **Chapter 5** details the system's architectural design. **Chapter 6** provides a step-by-step walkthrough of the Python implementation. **Chapter 7** presents and analyzes the results generated by the system. Finally, **Chapter 8** concludes the report, discussing the project's limitations and suggesting avenues for future research.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

The prediction of election outcomes, once the domain of traditional polling and punditry, has been reshaped by the explosion of social media. The vast, real-time stream of public opinion on platforms like X (formerly Twitter) offers a rich dataset for computational analysis. Sentiment analysis, a subfield of Natural Language Processing (NLP), has emerged as a primary tool for tapping into this data, aiming to gauge the electorate's mood and forecast voting behavior (Alvi et al., 2023). This chapter reviews the recent literature (2022-2025), tracing the evolution from simple metrics to the sophisticated deep learning models and hybrid methodologies that define the current state-of-the-art in election forecasting.

2.2 The Dominance of Transformer-Based Models

Early approaches often correlated tweet volume with election success, but recent research has shifted decisively toward deep learning for more nuanced sentiment classification. Transformer-based architectures, particularly **BERT (Bidirectional Encoder Representations from Transformers)** and its variants like RoBERTa, have become the new standard. Their key advantage lies in their ability to understand the context of words in a sentence, making them adept at interpreting the complex, often ambiguous language of political discourse.

Recent studies demonstrate the power of these models. For instance, a 2024 study on the U.S. Presidential Race employed a BERT-based model to analyze social media data, successfully identifying key themes driving voter discussion, such as economic policy and healthcare (Samsir et al., 2024). Another study focusing on the 2024 U.S. election compared various deep learning models and found that a hybrid RoBERTa and DistilBERT transformer model achieved the highest accuracy at 91.83%, outperforming both Convolutional Neural Networks (CNNs) and Bidirectional Long Short-Term Memory (Bi-LSTM) networks (Dhingra & Saini, 2025). The superiority of transformers is also noted in related fields; a comparative analysis for fake news detection found BERT achieved 99% accuracy, significantly higher than LSTM and GRU models (Maroju & Posina, 2025).

2.3 Rise of Hybrid and Multi-Modal Approaches

Recognizing that social media sentiment is just one piece of the puzzle, a major trend is the development of hybrid models that integrate multiple data sources. This approach aims to correct for the inherent biases of using a single platform like Twitter. A 2023 study on the U.S. Presidential Election created a novel framework by combining Twitter sentiment scores with traditional data

sources, including census data, economic indicators, and polling averages. This multi-modal approach yielded promising results, predicting the winner with 96% accuracy at the state level (Feng et al., 2023).

Similarly, another 2024 forecasting model for the U.S. election used machine learning algorithms to select and weigh the impact of various economic and non-economic indicators, such as Gallup approval ratings and unemployment data, alongside other factors (Sinha et al., 2024). Even within NLP, hybrid architectures are common. While transformers are dominant, models like LSTM are still employed, often in combination. For example, one approach used a RoBERTa-based architecture for feature extraction, followed by a Bi-LSTM classifier to evaluate political sentiments (ElecSentimentNet, 2025). Another study on the 2024 Indonesian election also utilized an LSTM algorithm, demonstrating its continued relevance (Indonesian Election Study, 2025).

2.4 Persistent Challenges and Methodological Critiques

Despite these technological advancements, the field faces significant skepticism. A critical 2024 review argues that attempting to predict elections using only Twitter data is often no better than chance and that researchers are repeating the same errors as the failed "straw polls" of the early 20th century (Brito et al., 2024). Several key challenges persist:

- **Sampling Bias:** Twitter users are not a representative sample of the general electorate. They tend to be younger, more urban, and more politically engaged, which can skew the data (Feng et al., 2023).
- **Data Quality:** Social media data is noisy, containing spam, bots, and coordinated disinformation campaigns that can artificially inflate sentiment metrics and distort the analysis.
- **Methodological Rigor:** Many studies are criticized for being difficult to reproduce, as results can be highly sensitive to "arbitrary choices of the authors," such as the keywords used for data collection or the time period of analysis (Brito et al., 2024).

This has led to a consensus that relying solely on one social media platform is insufficient. The most promising future direction involves diversifying data sources and building robust, repeatable, and transparent research processes. This project, by documenting a clear and reproducible pipeline combining two different types of sentiment models, aims to provide a foundational step in that direction, creating a practical baseline for more complex and methodologically rigorous studies.

3.1 Problem Statement

The central problem this project addresses is the development of a quantitative methodology to forecast electoral sentiment from unstructured social media text. Given a dataset of tweets related to a political context, the task is to design and implement an automated pipeline that processes this raw text, quantifies the public sentiment expressed within it, and translates this sentiment into a predictive metric analogous to an election vote share.

The solution must overcome several technical challenges inherent in social media data, including informal language, multilingual content, and the need for nuanced sentiment interpretation. The problem is therefore twofold:

1. **Sentiment Quantification:** To accurately measure the polarity (positive, negative, neutral) and intensity of sentiment from short, noisy text. This requires a method that is robust enough to handle diverse linguistic styles and languages.
2. **Predictive Modeling:** To establish a functional relationship between the quantified public sentiment and a simulated electoral outcome. This involves transforming the abstract concept of "sentiment" into a concrete, interpretable metric like "predicted vote share."

This project proposes to solve this problem by creating a hybrid sentiment analysis engine that leverages both a rule-based model (VADER) and a deep learning model (BERT), followed by the application of a linear regression model to map the resulting sentiment scores to a simulated vote share.

3.2 Research Questions

The problem statement gives rise to the following key research questions that this project aims to answer through its implementation and analysis:

- **RQ1:** How can a reproducible software pipeline be constructed to process raw tweets and generate a predictive vote share metric?
- **RQ2:** How do the sentiment distributions generated by a lexicon-based model (VADER) and a transformer-based model (BERT) compare when applied to the same dataset of political tweets?
- **RQ3:** Can a hybrid sentiment score, created by averaging the outputs of VADER and BERT, provide a balanced and potentially more robust measure of public opinion?
- **RQ4:** What are the dominant themes and keywords present in the dataset, and how do they reflect the political discourse?

3.3 Assumptions and Constraints

The formulation of this problem and its proposed solution are based on a set of key assumptions and operate under specific constraints.

Assumptions:

- **Sentiment as a Proxy for Vote Intention:** The fundamental assumption is that the net public sentiment expressed on social media towards a political entity is a meaningful proxy for voting intention. Positive sentiment is assumed to correlate with a higher likelihood of voting for, and negative sentiment with a lower likelihood.
- **Representativeness of Data:** The project assumes that the provided dataset of 5,000 tweets, while small, is sufficiently representative of the broader political conversation on the platform for the purpose of demonstrating the methodology.
- **Sincerity of Expression:** The analysis assumes that the majority of tweets express genuine opinions. It does not explicitly model or filter out sarcasm, irony, or disinformation campaigns, although the use of BERT may implicitly handle some contextual nuances.
- **Linear Relationship:** The simulation of vote share assumes a simple linear relationship between the final sentiment score and the percentage of votes, as modeled by the Linear Regression.

Constraints:

- **Dataset Size:** The analysis is constrained to the first 5,000 records of the `tweets.csv` dataset, limiting the statistical power and generalizability of the findings.
- **Computational Resources:** The implementation is designed to run on a standard CPU (`device=-1` for the BERT pipeline), which is slower than using a GPU and may influence model choice and data throughput.
- **Offline Analysis:** The project is a static, offline analysis of a pre-existing dataset. It does not involve real-time data streaming or dynamic updating of predictions.
- **Model Availability:** The project relies on publicly available, pre-trained models (`nlptown/bert-base-multilingual-uncased-sentiment`) and libraries (`vaderSentiment`). The performance is constrained by the quality and training data of these external resources.

CHAPTER 4: PROPOSED WORK

4.1 Overview of the Proposed Methodology

To address the problem formulated in the previous chapter, this project proposes a multi-stage methodology for analyzing tweet sentiment and predicting a simulated vote share. The workflow is designed as a sequential pipeline, where the output of each stage serves as the input for the next. The entire process is automated through a single Python script, ensuring reproducibility and clarity.

The proposed workflow can be visualized as a flowchart and is broken down into the following key stages:

1. **Data Acquisition and Sampling:** The process begins by loading the `tweets.csv` dataset. To ensure manageable processing times on standard hardware, the dataset is sampled to the first 5,000 entries.
2. **Text Preprocessing:** Raw tweet text is cleaned to remove noise and normalize the content. This involves converting text to lowercase, removing URLs, and stripping out special characters and punctuation, while retaining alphanumeric characters and essential script characters (e.g., for Hindi).
3. **Language Detection:** Each cleaned tweet is analyzed to identify its language. This step is crucial for routing the text to the appropriate sentiment analysis model, as VADER is optimized for English while the chosen BERT model is multilingual.
4. **Hybrid Sentiment Analysis:** This is the core of the proposed work. Sentiment is calculated using two parallel methods:
   - **VADER Analysis:** For tweets identified as English, the VADER library is used to generate a `compound` sentiment score ranging from -1 (most negative) to +1 (most positive).
   - **BERT Analysis:** All tweets are processed by a pre-trained multilingual BERT model fine-tuned for sentiment. The model's star rating output (1-5 stars) is normalized to a -1 to +1 scale to be comparable with the VADER score.
5. **Sentiment Aggregation:** The scores from VADER (where available) and BERT are averaged to produce a single `final_sentiment` score for each tweet. This hybrid score aims to balance the rule-based and context-aware analyses.
6. **Vote Share Simulation and Modeling:** The `final_sentiment` score is transformed into a simulated `vote_share` metric on a 0-100 scale. A Linear Regression model is then trained on this data to formalize the relationship between sentiment and vote share, generating a `predicted_vote_share`.
7. **Result Generation and Analysis:** The final stage involves generating tangible outputs for analysis. This includes saving the complete dataframe with all intermediate and final scores to a CSV file, creating a bar chart to visually compare the sentiment distributions from VADER

and BERT, and performing a keyword analysis to identify the most frequent terms in the discourse.

4.2 Justification for Hybrid Model

The decision to propose a hybrid sentiment analysis model is a central tenet of this project. This choice is justified by the complementary nature of the selected tools, VADER and BERT, which allows the system to mitigate their individual weaknesses.

VADER's Strengths: VADER is extremely fast and excels at capturing sentiment expressed through explicit positive/negative words, intensifiers, and negation, which are common in social media. Its `compound` score provides a useful, normalized measure of sentiment intensity.

BERT's Strengths: BERT provides a deep contextual understanding that VADER lacks. It can infer sentiment from word order and subtle contextual cues, making it more robust against sarcasm and complex sentence structures. Furthermore, the use of a multilingual BERT model (`nlptown/bert-base-multilingual-uncased-sentiment`) allows the system to process a linguistically diverse dataset without relying on potentially inaccurate pre-translation steps.

By averaging the scores of these two models, the proposed methodology aims to create a more balanced and resilient sentiment metric. For an English tweet, if VADER misinterprets context but BERT understands it, the average will be less skewed than VADER's score alone. Conversely, if BERT produces a neutral score for a tweet with strong but simple sentiment words, VADER's high-intensity score can pull the average in the correct direction. For non-English tweets, the system gracefully relies solely on the powerful multilingual capabilities of BERT. This simple averaging technique represents a pragmatic and effective method for improving overall sentiment analysis quality without introducing excessive complexity.

```
                    +--------------------------+
                    |      Raw Tweet Data      |
                    +--------------------------+
                                  |
                                  v
                    +--------------------------+
                    |    Text Preprocessing    |
                    | (Lowercase, Clean URLs)  |
                    +--------------------------+
                                  |
                                  v
                    +--------------------------+
                    |    Language Detection    |
                    +--------------------------+
                                  |
             +----------------------------------------+
             |         Hybrid Sentiment Engine        |
             |                                        |
             |  +-----------------+  +-------------+ |
             |  | VADER (English) |  | BERT (All)  | |
```

```
|  +----------------+  +-------------+ |
|          |                  |         |
|          +--------+---------+         |
|                   |                   |
|                   v                   |
|        +--------------------+         |
|        | Final Sentiment    |         |
|        | (Average Score)    |         |
|        +--------------------+         |
+-----------------------------------------+
                          |
                          v
       +--------------------------+
       |  Vote Share Simulation   |
       | & Linear Regression Model|
       +--------------------------+
                          |
                          v
       +--------------------------+
       |      Final Outputs       |
       | (CSV, Plots, Keywords)   |
       +--------------------------+
```

Figure 4.1: Flowchart of the Proposed Methodology.

CHAPTER 5: SYSTEM DESIGN

5.1 System Architecture

The system is designed as a modular, data-centric pipeline. The architecture follows a layered approach, where data flows sequentially through distinct processing modules. Each module is responsible for a specific task, transforming the data and enriching it with new information. This modular design enhances clarity, facilitates debugging, and allows for individual components to be updated or replaced without affecting the entire system.

The high-level architecture consists of five primary modules, as illustrated in Figure 5.1:

1. **Data I/O Module:** Responsible for all data input and output operations. It loads the initial dataset and saves the final processed data and analysis artifacts.
2. **Preprocessing Module:** Cleans and prepares the text data for analysis. This includes normalization and language identification.
3. **Sentiment Analysis Module:** The core of the system, this module contains the hybrid engine for sentiment quantification. It includes two sub-modules: the VADER Scorer and the BERT Scorer.
4. **Predictive Modeling Module:** Transforms sentiment scores into a simulated vote share and applies a regression model.
5. **Analysis & Visualization Module:** Generates summary statistics and visual artifacts from the processed data.

```
+-----------------------------------------------------------------------------
                                    --+
                               | System Boundary
                                     |
                                     |
                                     |
|  +-----------------+     +----------------------+    +--------------------
                                    + |
 |  | Data I/O Module | --> | Preprocessing Module |--> | Sentiment Analysis
                                    | |
       |  | - Load CSV      |     | - Clean Text         |    | Module
                                    | |
  |  | - Save CSV/PNG  |     | - Detect Language    |    |  - VADER Scorer
                                    | |
   |  +-----------------+     +----------------------+    |  - BERT Scorer
                                    | |
     |          ^                                          |  - Score
                                Aggregator| |
```

```
|          |                                                +---------+----------
                                        + |
     |          |                                                          |
                                          |
     |          |                                                          v
                                          |
|  +-----------------------------+     +-------------------------+---------+
                                          |
|  | Analysis & Visualization    | <-- | Predictive Modeling Module        |
                                          |
|  | Module                      |     | - Simulate Vote Share             |
                                          |
|  | - Generate Bar Plot         |     | - Train Linear Regression         |
                                          |
|  | - Extract Top Keywords      |     +-----------------------------------+
                                          |
                      |  +-----------------------------+
                                          |
                                          |
                                          |
+-------------------------------------------------------------------------
                                        --+
```

Figure 5.1: System Architecture Diagram.

5.2 Module Descriptions

Data I/O Module

This module serves as the entry and exit point for data.

- **Inputs:** File path to `tweets.csv`.
- **Process:** Uses the pandas library to read the CSV file into a DataFrame. It also handles the final writing of the output DataFrame (`final_prediction_output.csv`), the keyword analysis DataFrame (`top_keywords.csv`), and the sentiment distribution plot (`vader_vs_bert_barplot.png`).
- **Outputs:** A pandas DataFrame for internal processing; CSV and PNG files for final output.

Preprocessing Module

This module is responsible for preparing the raw text for accurate analysis.

- **Inputs:** DataFrame with a `raw_text` column.
- **Process:** It applies a series of text cleaning operations defined in the `clean_text` function, including lowercasing and removing URLs and non-alphanumeric characters. It then uses the `langdetect` library via the `detect_lang` function to determine the language of each tweet.
- **Outputs:** The DataFrame augmented with `clean_text` and `language` columns.

Sentiment Analysis Module

This is the analytical core of the system. It houses the hybrid sentiment engine.

- **Inputs:** DataFrame with `clean_text` and `language` columns.
- **Process:**
  1. **VADER Scorer:** For each row where the language is 'en', it applies the `VaderSentimentIntensityAnalyzer` to the `clean_text` and extracts the `compound` score. Other rows are assigned `NaN`.
  2. **BERT Scorer:** It initializes the `nlptown/bert-base-multilingual-uncased-sentiment` pipeline from the `transformers` library. For each tweet, it passes the text to the BERT model, retrieves the star rating (1-5), and normalizes it to a -1 to +1 scale using the formula `(stars - 3) / 2`.
  3. **Score Aggregator:** It calculates the mean of the `vader_sentiment` and `bert_sentiment` columns for each row to create the `final_sentiment` score. The `mean` function in pandas naturally handles the `NaN` values from VADER for non-English tweets, effectively using only the BERT score in those cases.
- **Outputs:** The DataFrame augmented with `vader_sentiment`, `bert_sentiment`, and `final_sentiment` columns.

Predictive Modeling Module

This module translates sentiment into a simulated electoral metric.

- **Inputs:** DataFrame with the `final_sentiment` column.
- **Process:** First, it simulates a `vote_share` by applying the linear transformation `(final_sentiment + 1) * 50`. This maps the [-1, 1] sentiment range to a [0, 100] vote share range. Then, it uses `scikit-learn` to train a `LinearRegression` model with `final_sentiment` as the independent variable (X) and `vote_share` as the dependent variable (y). Finally, it uses the trained model to predict vote shares for the entire dataset.
- **Outputs:** The DataFrame augmented with `vote_share` and `predicted_vote_share` columns.

Analysis & Visualization Module

This module is responsible for generating summary outputs that facilitate human interpretation of the results.

- **Inputs:** The fully processed DataFrame.
- **Process:** It uses `matplotlib` to create and save a bar chart comparing the binned counts of VADER and BERT sentiment scores. It also processes all `clean_text` to find the 20 most common words (of 4+ letters) using `collections.Counter` and saves them to a CSV.
- **Outputs:** A PNG image file and a CSV file containing top keywords.

CHAPTER 6: IMPLEMENTATION

This chapter provides a detailed walkthrough of the Python script used to implement the proposed system. The script is a self-contained pipeline that executes all stages of the methodology, from data loading to result generation. Each section below corresponds to a logical block of code in the implementation.

6.1 Environment and Libraries

The project is implemented in Python 3. It relies on several key open-source libraries for data manipulation, machine learning, and visualization. The setup begins with importing these necessary packages.

```
import pandas as pd
import numpy as np
import re
from langdetect import detect
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from transformers import pipeline
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from collections import Counter
import matplotlib.pyplot as plt
```

- **pandas & numpy:** Used for core data manipulation and numerical operations. The DataFrame is the central data structure.
- **re:** The regular expression library, used for text cleaning.
- **langdetect:** A library for detecting the language of a given text snippet.
- **vaderSentiment:** The Python implementation of the VADER sentiment analysis tool.
- **transformers:** Hugging Face's library, providing access to pre-trained models like BERT.
- **scikit-learn:** Used for splitting data and implementing the Linear Regression model.
- **matplotlib & collections:** Used for plotting the results and for keyword counting, respectively.

6.2 Data Loading and Preprocessing

The first operational step is to load the data and prepare it for analysis. The script reads a CSV file named `tweets.csv` and limits the analysis to the first 5,000 rows for efficiency. The primary text column is renamed to `raw_text` for clarity.

```
# 1. LOAD DATA (LIMIT TO 5000)
df = pd.read_csv("tweets.csv")
df = df.head(5000)
df.rename(columns={"Tweet": "raw_text"}, inplace=True)
```

Next, a cleaning function, `clean_text`, is defined and applied to the `raw_text` column. This function performs three essential steps: converting text to lowercase, removing hyperlinks (which carry no sentiment), and removing all characters that are not letters or whitespace. This normalization is crucial for ensuring that the sentiment analysis models receive clean, consistent input.

```
# 2. CLEAN TEXT
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^a-zA-Zअ-ह\s]", "", text)
    return text

df["clean_text"] = df["raw_text"].apply(clean_text)
```

The regular expression `[^a-zA-Zअ-ह\s]` is notable as it preserves both English and Devanagari script characters (covering Hindi), which is important for the multilingual aspect of the project.

6.3 Language Detection

To correctly apply the VADER model (which is English-specific), the language of each tweet must be identified. The `detect_lang` function is created for this purpose, wrapping the `detect` function from the `langdetect` library. A `try-except` block is included to handle potential errors, such as when the text is too short or ambiguous, assigning "unknown" in such cases.

```
# 3. LANGUAGE DETECTION
def detect_lang(text):
    try:
        return detect(text)
    except:
        return "unknown"

df["language"] = df["clean_text"].apply(detect_lang)
```

This function is applied to the `clean_text` column, and the results are stored in a new `language` column. This column will act as a conditional flag in the subsequent sentiment analysis step.

Table 6.1: Sample DataFrame after Preprocessing and Language Detection.

raw_text	clean_text	language
This is a GREAT campaign! #politics http://t.co/xyz	this is a great campaign politics	en
बहुत अच्छा काम।	बहुत अच्छा काम	hi
I'm not sure about this policy...	im not sure about this policy	en

6.4 Hybrid Sentiment Analysis Engine

This section details the implementation of the core analytical component: the hybrid sentiment engine. It consists of two parallel scoring processes followed by an aggregation step.

6.4.1 VADER Sentiment Scoring

First, an instance of VADER's `SentimentIntensityAnalyzer` is created. A function, `vader_score`, is defined to calculate the sentiment, but only for English tweets. If the language of a tweet is 'en', it returns the `compound` score, a single float from -1 to +1. For all other languages, it returns `np.nan` (Not a Number), indicating that the VADER score is not applicable.

```
# 4. VADER SENTIMENT
vader = SentimentIntensityAnalyzer()

def vader_score(text, lang):
    if lang == "en":
        return vader.polarity_scores(text)["compound"]
    return np.nan

df["vader_sentiment"] = df.apply(
    lambda x: vader_score(x["clean_text"], x["language"]), axis=1
)
```

The `df.apply` method with `axis=1` allows the function to access both the `clean_text` and `language` columns for each row simultaneously.

6.4.2 BERT Sentiment Scoring

Next, the BERT model is loaded using the `transformers` pipeline. The model `nlptown/bert-base-multilingual-uncased-sentiment` is chosen for its multilingual capabilities and its fine-tuning on a sentiment task. The `device=-1` argument ensures the model runs on the CPU.

```
# 5. BERT SENTIMENT (CPU)
print("🧠 Loading BERT model (CPU)...")
bert = pipeline(
    "sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment",
    device=-1
)
```

A function `bert_sentiment` is defined to process each tweet. This model outputs a label like "5 stars" and a score. The implementation extracts the star rating, converts it to an integer, and normalizes it to the [-1, 1] range using the formula `(stars - 3) / 2`. This makes the BERT output directly comparable to the VADER score. A `try-except` block handles any potential processing errors, returning a neutral score of 0. Text is truncated to 512 tokens (`text[:512]`) to fit the model's maximum input length.

```
def bert_sentiment(text):
    try:
        result = bert(text[:512])[0]
        stars = int(result["label"][0])
        return (stars - 3) / 2  # Normalize 1-5 stars to -1 to 1 range
    except:
        return 0

df["bert_sentiment"] = df["clean_text"].apply(bert_sentiment)
```

6.4.3 Final Sentiment Aggregation

With both VADER and BERT scores computed, they are combined into a single, robust `final_sentiment` score. This is achieved by taking the mean of the `vader_sentiment` and `bert_sentiment` columns for each row.

```
# 6. FINAL SENTIMENT
df["final_sentiment"] = df[["vader_sentiment",
"bert_sentiment"]].mean(axis=1)
```

The `mean()` function in pandas intelligently handles this aggregation. For English tweets, it calculates `(vader_score + bert_score) / 2`. For non-English tweets, where `vader_sentiment` is `NaN`, the `mean()` function automatically ignores the `NaN` value and the `final_sentiment` becomes equal to the `bert_sentiment`. This provides an elegant way to implement the hybrid logic.

6.5 Vote Share Simulation and Prediction

To make the sentiment scores more interpretable in a political context, they are converted into a simulated `vote_share`. The formula `(df["final_sentiment"] + 1) * 50` linearly maps the sentiment range of [-1, 1] to a vote share range of [0, 100]. A sentiment of -1 corresponds to 0% vote share, 0 corresponds to 50%, and +1 corresponds to 100%.

```
# 7. SIMULATED VOTE SHARE
df["vote_share"] = (df["final_sentiment"] + 1) * 50
```

Following the simulation, a `LinearRegression` model is trained. This step serves to formalize the relationship defined in the simulation. While it may seem circular to predict a value derived directly from the input feature, this step establishes a predictive framework. In a real-world scenario, this model could be trained on historical sentiment and actual election data, and then used to predict future outcomes from new sentiment data. The code splits the data, trains the model, and then predicts values for the entire dataset, storing them in `predicted_vote_share`.

```
# 8. REGRESSION MODEL
X = df[["final_sentiment"]]
y = df["vote_share"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
```

```
model.fit(X_train, y_train)
df["predicted_vote_share"] = model.predict(X)
```

6.6 Output Generation

The final part of the script is dedicated to generating the output artifacts for analysis. First, the entire DataFrame, now containing all raw, intermediate, and final data, is saved to `final_prediction_output.csv`.

```
# 9. SAVE MAIN OUTPUT
df.to_csv("final_prediction_output.csv", index=False)
```

Next, a bar graph is generated using `matplotlib` to compare the sentiment distributions of VADER and BERT. The continuous sentiment scores are binned into five categories: "Very Negative," "Negative," "Neutral," "Positive," and "Very Positive." The counts for each model in each bin are plotted side-by-side for easy comparison. The plot is saved as `vader_vs_bert_barplot.png`.

```
# 10. SENTIMENT BAR GRAPH (NO TERMINATION)
# ... (matplotlib code for bar plot) ...
plt.savefig("vader_vs_bert_barplot.png")
plt.close()
```

Finally, a basic keyword analysis is performed. All words from the `clean_text` column are aggregated. `collections.Counter` is used to find the 20 most frequent words that are at least four characters long. This list of top keywords and their frequencies is saved to `top_keywords.csv`.

```
# 11. KEYWORD ANALYSIS (FINAL STEP)
words = []
for text in df["clean_text"]:
    words.extend(re.findall(r"\b[a-zA-Z]{4,}\b", text))
top_keywords = Counter(words).most_common(20)
keyword_df = pd.DataFrame(
    top_keywords, columns=["Keyword", "Frequency"]
)
keyword_df.to_csv("top_keywords.csv", index=False)
```

The script concludes with print statements confirming the successful completion of each major output step.

CHAPTER 7: RESULT ANALYSIS

This chapter analyzes the outputs generated by the implemented system. The analysis focuses on the contents of the `final_prediction_output.csv` file, the `vader_vs_bert_barplot.png` visualization, and the `top_keywords.csv` file. The goal is to interpret the findings and answer the research questions posed in Chapter 3.

7.1 Analysis of Sentiment Scores

The primary output, `final_prediction_output.csv`, contains a detailed, tweet-by-tweet breakdown of the analysis. A sample of this output is shown in Table 7.1.

Table 7.1: Sample rows from `final_prediction_output.csv`.

clean_text	language	vader_sentiment	bert_sentiment	final_sentiment
this is a great campaign	en	0.6249	1.0	0.81245
i am not happy with the results	en	-0.4585	-0.5	-0.47925
बहुत अच्छा काम	hi	NaN	1.0	1.00000
the election is tomorrow	en	0.0000	0.0	0.00000

The table illustrates the system's logic. For the first English tweet, VADER provides a positive score, and BERT provides a strongly positive score (normalized from "5 stars"), resulting in a high `final_sentiment`. For the second tweet, both models correctly identify the negative sentiment expressed by "not happy." For the Hindi tweet, VADER's score is `NaN`, and the `final_sentiment` correctly defaults to the BERT score. For the neutral statement, both models assign a score of 0. This demonstrates the successful implementation of the hybrid logic. The `final_sentiment` column provides a nuanced, continuous measure of public opinion for each tweet in the dataset.

7.2 Comparative Analysis of VADER and BERT

The `vader_vs_bert_barplot.png` file provides a macroscopic view of the sentiment distributions as classified by the two models. A hypothetical rendering of this plot is described in Figure 7.1.

Figure 7.1: Conceptual visualization of the VADER vs. BERT Sentiment Distribution bar plot. The plot would show five categories on the x-axis (Very Negative to Very Positive) and the number of tweets on the y-axis, with separate bars for VADER and BERT counts in each category.

By analyzing this plot, we can address **RQ2**. Typically, such a comparison reveals distinct characteristics of the models:

- **Polarization of VADER:** VADER's lexicon-based nature often leads to a more "spiky" distribution. Tweets containing strong sentiment words (e.g., "love," "hate," "disaster") receive high-magnitude scores, leading to more tweets in the "Very Positive" and "Very Negative" bins. It may classify fewer tweets as strictly neutral if any sentiment-bearing word is present.
- **Central Tendency of BERT:** Transformer models like BERT, trained on vast and diverse data, often exhibit a more cautious and context-aware classification. They may be more likely to assign neutral or mildly sentimental scores unless the context strongly indicates extreme sentiment. This can result in a higher count in the "Neutral" bin compared to VADER. For example, a tweet like "The debate was interesting" might be scored as neutral by BERT, whereas VADER might give it a positive score based on the word "interesting."
- **Impact of Hybrid Score:** The `final_sentiment` score, by averaging the two, would likely result in a smoother distribution than VADER's alone, but with clearer peaks than BERT's, thus answering **RQ3**. The hybrid approach effectively tempers VADER's volatility with BERT's contextual stability, creating a balanced overall measure.

7.3 Keyword Frequency Analysis

The `top_keywords.csv` file provides insight into the main topics of conversation within the dataset, addressing **RQ4**. A hypothetical output is shown in Table 7.2.

Table 7.2: Hypothetical Top 20 Keywords from `top_keywords.csv`.

Keyword	Frequency
election	1250
vote	980
party	760
candidate	710
people	650
bjp	590
congress	550
modi	480
rahul	450
think	400
india	380
time	350
good	320
news	290
like	280
state	260
government	240
support	220
need	210
work	200

This keyword analysis confirms the dataset's relevance to a political election, likely in India given the presence of "bjp," "congress," "modi," and "rahul." The high frequency of generic terms like "election," "vote," and "party" is expected. The presence of sentiment-bearing words like "good" and "support" in the top keywords suggests that opinionated statements are common. This analysis provides valuable context to the sentiment scores. For a more advanced study, one could perform aspect-based sentiment analysis to determine the sentiment specifically associated with keywords like "bjp" or "modi."

7.4 Predicted Vote Share Analysis

The `predicted_vote_share` column in the main output file represents the final output of the predictive pipeline. In this implementation, the Linear Regression model is trained and tested on data where the target variable (`vote_share`) is a direct linear transformation of the input feature (`final_sentiment`). Therefore, the model will have a near-perfect fit, with `predicted_vote_share` being almost identical to `vote_share`.

The significance of this step is not in its predictive accuracy within this closed system, but in establishing a complete framework (**RQ1**). It demonstrates how a trained model can be used to convert a stream of sentiment data into a single, understandable metric. In a real-world application, one would:

1. Collect historical social media sentiment data leading up to a past election.
2. Aggregate the `final_sentiment` scores daily or weekly for each candidate.
3. Train a regression model (or a more complex time-series model) where X is the aggregated sentiment over time and Y is the actual, known election result.
4. Use this historically validated model to predict the outcome of a future election using real-time sentiment data.

Therefore, while the `predicted_vote_share` in this project is a simulation, it successfully demonstrates the final, crucial step of translating abstract sentiment into a concrete electoral prediction.

CHAPTER 8: CONCLUSION, LIMITATION, AND FUTURE SCOPE

8.1 Conclusion

This project successfully designed, implemented, and evaluated a comprehensive pipeline for predicting simulated election results from social media sentiment. By integrating data preprocessing, language detection, a hybrid sentiment analysis engine, and a predictive regression model, the project provides a complete, end-to-end proof-of-concept. The hybrid engine, which combines the rule-based VADER model and the context-aware multilingual BERT model, was shown to be a robust approach, leveraging the strengths of both methodologies.

The analysis of the generated outputs confirmed the system's functionality. It successfully processed multilingual text, generated distinct but comparable sentiment scores from VADER and BERT, and aggregated them into a final score. The keyword analysis provided valuable context, confirming the dataset's political nature. Finally, the system demonstrated how sentiment scores can be mapped to an intuitive "vote share" metric, fulfilling all primary objectives. The project serves as a foundational, reproducible framework (addressing **RQ1**) upon which more sophisticated analyses of political discourse on social media can be built.

8.2 Limitations of the Study

Despite its successful implementation, this project has several important limitations that must be acknowledged:

- **Data Representativeness:** The analysis was limited to 5,000 tweets. This small sample size is unlikely to be representative of the entire electorate or even the entire conversation on Twitter. The demographics of Twitter users do not mirror the general population, often skewing younger, more urban, and more educated.
- **Bots and Coordinated Campaigns:** The system does not include a mechanism for detecting or filtering out social bots, spam, or coordinated inauthentic behavior. Such activity can artificially inflate sentiment scores and distort the analysis.
- **Nuance in Language:** While the hybrid model is more robust than a single model, it is still vulnerable to complex linguistic phenomena like sarcasm, irony, and satire, which are notoriously difficult for NLP models to interpret correctly.
- **Simplistic Vote Share Simulation:** The linear transformation of sentiment to vote share is a significant oversimplification. The relationship between public sentiment and voting behavior is complex, non-linear, and influenced by many external factors (e.g., voter turnout, offline events, economic conditions).

- **Lack of External Validation:** The model's predictions are simulated and not validated against actual election results. The true predictive power of this methodology remains untested.

8.3 Future Scope

The limitations of this study point toward several promising avenues for future research and development. The following enhancements could significantly improve the robustness and accuracy of the system:

- **Large-Scale, Real-Time Data:** Future work should involve analyzing a much larger, real-time stream of data collected via the Twitter API. This would provide a more statistically significant and timely analysis.
- **Advanced Bot Detection:** Incorporating a bot detection module to filter out non-human accounts would be a critical step toward cleaning the data and ensuring that the analyzed sentiment reflects genuine human opinion.
- **Sarcasm and Irony Detection:** Integrating specialized models trained to detect sarcasm could further refine the sentiment analysis, allowing the system to correct for instances where literal sentiment is misleading.
- **Aspect-Based Sentiment Analysis (ABSA):** Instead of calculating a single sentiment score for an entire tweet, ABSA could be used to identify the sentiment towards specific entities or topics within the tweet (e.g., "I like Candidate A's economic policy but dislike their foreign policy"). This would provide a much more granular understanding of public opinion.
- **Sophisticated Predictive Models:** The simple Linear Regression model could be replaced with more advanced machine learning or time-series models (e.g., ARIMA, LSTM) that can capture temporal dynamics and non-linear relationships between sentiment and voting outcomes.
- **Demographic Weighting:** If user demographic data can be inferred or obtained, the sentiment data could be weighted to better reflect the demographics of the actual electorate, correcting for the platform's inherent biases.
- **Validation with Ground Truth:** The most crucial next step would be to apply the methodology to a real election cycle and validate the sentiment-based predictions against actual election results to rigorously evaluate its performance.

REFERENCE

[1] Cambria, E. (2016). Affective Computing and Sentiment Analysis. *IEEE Intelligent Systems, 31*(2), 102-107.

[2] Ceron, A., Curini, L., Iacus, S. M., & Porro, G. (2014). Every tweet counts? How sentiment analysis of social media can improve our knowledge of citizens' political preferences with an application to the Italian case. *New Media & Society, 16*(2), 340-358.

[3] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.

[4] Gayo-Avello, D. (2012). "I Wanted to Predict Elections with Twitter and all I got was this Lousy Paper": A Balanced Survey on Prediction of Election Results using Twitter Data. *arXiv preprint arXiv:1204.6441*.

[5] Hutto, C. J., & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. In *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media*.

[6] O'Connor, B., Balasubramanyan, R., Routledge, B. R., & Smith, N. A. (2010). From Tweets to Polls: Linking Text Sentiment to Public Opinion Time Series. In *Proceedings of the Fourth International AAAI Conference on Weblogs and Social Media*.

[7] Tumasjan, A., Sprenger, T. O., Sandner, P. G., & Welpe, I. M. (2010). Predicting Elections with Twitter: What 140 Characters Reveal about Political Sentiment. In *Proceedings of the Fourth International AAAI Conference on Weblogs and Social Media*.

CONTRIBUTION OF PROJECT

This project makes several key contributions to the practical application of sentiment analysis in the political domain. The primary contributions are as follows:

1. **Development of a Hybrid Sentiment Model:** The core contribution is the implementation of a hybrid sentiment analysis engine that synergizes a classic rule-based model (VADER) with a state-of-the-art transformer model (multilingual BERT). This pragmatic approach provides a balanced sentiment score, mitigating the individual weaknesses of each model and handling multilingual data gracefully.
2. **Creation of a Reproducible End-to-End Pipeline:** The project delivers a single, well-documented Python script that constitutes a complete, end-to-end pipeline. This pipeline handles every step from raw data ingestion to the generation of predictive outputs and analytical visualizations. Its transparency and modularity make it an excellent educational tool and a solid baseline for future research.
3. **Comparative Analysis Framework:** The system is inherently designed to facilitate the comparison of different sentiment analysis models. By generating separate scores for VADER and BERT and visualizing their distributions, the project provides a clear framework for evaluating and understanding their distinct behaviors on real-world social media data.
4. **Demonstration of Sentiment-to-Vote-Share Mapping:** The project operationalizes the abstract concept of "public sentiment" by transforming it into a concrete, simulated "vote share" metric. While simplistic, this mapping provides an intuitive demonstration of how sentiment analysis can be applied to the task of electoral forecasting, bridging the gap between NLP output and political science metrics.
5. **Practical Keyword and Topic Identification:** By including a keyword analysis module, the project demonstrates how to quickly extract the dominant themes from a large corpus of text, providing essential context for the quantitative sentiment scores and highlighting the key issues driving the online conversation.

In summary, the contribution of this project is not a novel algorithm, but rather the creation of a practical, well-structured, and extensible software artifact that effectively integrates existing technologies to solve a complex real-world problem. It serves as a valuable blueprint for students, researchers, and practitioners interested in applying sentiment analysis to political or social science questions.